NATURAL LANGUAGE PROCESSING LAB

Experiment 1: Tokenization, Stemming and Lemmatization

Apply various tokenization, stemming, and lemmatization on a given text. Compare the results in terms of word variations and semantic coherence.

Software: Python 3.x, spaCy, NLTK, Jupyter Notebook / VS Code

1. Background / Theory

3.1 Tokenization

Tokenization divides text into smaller units called tokens. Depending on the requirement, tokens may be sentences, words, punctuation marks, numbers, or other linguistic units.

In this experiment I will compare:

- Whitespace tokenization – splits text wherever whitespace occurs.
- spaCy word tokenization – uses linguistic rules and keeps punctuation as separate tokens.
- spaCy filtered tokenization – uses spaCy tokens but removes punctuation and spaces for downstream analysis.

3.2 Stemming

Stemming reduces a word to a crude root/stem by applying heuristic rules. The resulting stem does not necessarily have to be a valid English word. For example, connected, connecting and connection may be reduced to forms such as connect or connect-like stems depending on the algorithm.

Important implementation note: spaCy provides tokenization and lemmatization, but it does not provide a general-purpose built-in stemmer. Therefore, this manual uses spaCy as the main NLP library and NLTK's PorterStemmer and SnowballStemmer for the stemming part.

3.3 Lemmatization

Lemmatization maps a word to its dictionary/base form (lemma) using linguistic information. spaCy's lemmatizer can use the token's part of speech (POS) and the language model's linguistic rules. A lemma is generally more interpretable than an arbitrary stem.

3.4 Stemming vs. Lemmatization

Aspect	Stemming	Lemmatization	Expected observation
Method	Rule/heuristic based	Linguistic/dictionary based	Lemmatization is more language-aware
Output	May not be a real word	Usually a valid base word	Lemmas are easier to interpret
Speed	Usually faster	Usually more computationally expensive	Stemming can be faster

Meaning	May lose meaning	Usually preserves meaning better	Lemmatization should show better coherence

2. Requirements

- Python 3.9 or later (a recent Python 3.x version is recommended).
- Jupyter Notebook or VS Code with a Python environment.
- spaCy library.
- English spaCy model: en_core_web_sm.
- NLTK library for Porter and Snowball stemmers.
- Internet access only for first-time package/model installation, if required.

3. Setup and Installation

I will create a virtual environment so that the experiment dependencies remain isolated.

```
# Create a project folder
mkdir nlp_tokenization_experiment
cd nlp_tokenization_experiment

# Create a virtual environment
python -m venv .venv

# Windows
.venv\Scripts\activate

# Linux
source .venv/bin/activate

# Install libraries
python -m pip install --upgrade pip
pip install spacy nltk pandas jupyter

# Download the English spaCy model
python -m spacy download en_core_web_sm
```

If I am using Anaconda, I can create and activate the environment with conda instead. The important requirement is that the same environment is used to run the notebook.

4. Verify the Installation

```
import spacy
import nltk

print("spaCy version:", spacy.__version__)
print("NLTK version:", nltk.__version__)

nlp = spacy.load("en_core_web_sm")
print("spaCy English model loaded successfully.")
```

Expected result: the versions are displayed and the final message confirms that the model loaded successfully.

5. Experiment Workflow

1. Prepare the input paragraph.
2. Load the spaCy English model.
3. Apply whitespace tokenization.
4. Apply spaCy word tokenization.
5. Create a punctuation-filtered spaCy token list.
6. Apply Porter stemming.
7. Apply Snowball stemming.
8. Apply spaCy lemmatization.
9. Create a comparison table for selected words.
10. Measure vocabulary/word variation.
11. Evaluate semantic coherence qualitatively.
12. Write observations and conclusion.

6. Step 1 – Enter the Given Text

I will use the following text for the experiment. I may replace it with the text supplied by my instructor.

```
text = """
Natural language processing helps computers understand human language.
```

```
Students are studying, researchers are studying, and the models have learned
from many documents. A good NLP system analyzes words, sentences, and meanings.
The systems were designed to process, processed, and processing large collections
of text efficiently.
"""
```

7. Step 2 – Load spaCy

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

print("Number of spaCy tokens:", len(doc))
```

8. Step 3 – Perform Different Tokenization Methods

10.1 Whitespace Tokenization

```
whitespace_tokens = text.split()

print("Whitespace tokens:")
print(whitespace_tokens)
print("Count:", len(whitespace_tokens))
```

Observation: punctuation can remain attached to words, for example 'language.' may remain as one token.

10.2 spaCy Word Tokenization

```
spacy_tokens = [token.text for token in doc]

print("spaCy tokens:")
print(spacy_tokens)
print("Count:", len(spacy_tokens))
```

Observation: spaCy can separate punctuation from words. This is useful because downstream NLP tasks often need linguistic token boundaries rather than simple whitespace boundaries.

10.3 spaCy Tokens Without Punctuation/Spaces

```
clean_tokens = [
    token.text for token in doc
    if not token.is_space and not token.is_punct
]

print("Filtered tokens:")
print(clean_tokens)
print("Count:", len(clean_tokens))
```

9. Step 4 – Perform Stemming

11.1 Porter Stemmer

```
from nltk.stem import PorterStemmer

porter = PorterStemmer()

porter_stems = [
    porter.stem(token.text.lower())
    for token in doc
    if token.is_alpha
]

print(porter_stems)
```

11.2 Snowball Stemmer

```
from nltk.stem import SnowballStemmer

snowball = SnowballStemmer("english")

snowball_stems = [
    snowball.stem(token.text.lower())
    for token in doc
    if token.is_alpha
```

```
]

print(snowball_stems)
```

I will compare the two stemmers because different stemming algorithms can produce different word reductions.

10. Step 5 – Perform Lemmatization Using spaCy

```
lemmas = [
    token.lemma_.lower()
    for token in doc
    if token.is_alpha
]

print("spaCy lemmas:")
print(lemmas)
```

The spaCy model supplies linguistic information used to determine lemmas. The output should be interpreted as base forms rather than as arbitrary shortened strings.

11. Step 6 – Inspect POS and Lemmas Together

```
for token in doc:
    if token.is_alpha:
        print(
            f"{token.text:15} "
            f"POS={token.pos_:8} "
            f"TAG={token.tag_:8} "
            f"LEMMA={token.lemma_}"
        )
```

This step helps me explain why lemmatization is more linguistically informed. For example, the same spelling can receive different interpretations depending on context and POS.

12. Step 7 – Generate a Word-Level Comparison Table

```
import pandas as pd
```

```
comparison = pd.DataFrame({
    "Original": [token.text for token in doc if token.is_alpha],
    "POS": [token.pos_ for token in doc if token.is_alpha],
    "Porter Stem": [
        porter.stem(token.text.lower())
        for token in doc if token.is_alpha
    ],
    "Snowball Stem": [
        snowball.stem(token.text.lower())
        for token in doc if token.is_alpha
    ],
    "spaCy Lemma": [
        token.lemma_.lower()
        for token in doc if token.is_alpha
    ]
})

print(comparison.to_string(index=False))
```

I should include this table in my observation section. I should not only paste the complete output; I should discuss representative examples where the stem and lemma differ.

13. Step 8 – Compare Word Variations

```
# Unique alphabetic tokens
original_words = [
    token.text.lower()
    for token in doc
    if token.is_alpha
]

porter_words = [porter.stem(w) for w in original_words]
snowball_words = [snowball.stem(w) for w in original_words]

lemma_words = [
```

```
    token.lemma_.lower()
    for token in doc
    if token.is_alpha
]

metrics = pd.DataFrame({
    "Method": [
        "Original words",
        "Porter stemming",
        "Snowball stemming",
        "spaCy lemmatization"
    ],
    "Total words": [
        len(original_words),
        len(porter_words),
        len(snowball_words),
        len(lemma_words)
    ],
    "Unique forms": [
        len(set(original_words)),
        len(set(porter_words)),
        len(set(snowball_words)),
        len(set(lemma_words))
    ]
})

print(metrics)
```

A lower number of unique normalized forms indicates stronger vocabulary reduction, but lower is not automatically better. I must also check whether the normalized forms remain meaningful.

14. Step 9 – Evaluate Semantic Coherence

For this experiment, semantic coherence is primarily evaluated qualitatively. I will inspect whether the normalized forms are meaningful English words and whether related word forms are mapped consistently.

- Check whether the output is a valid or interpretable word.
- Check whether related forms such as study, studying and studied are reasonably normalized.
- Identify stems that look unnatural or truncated.
- Identify lemmas that preserve a recognizable dictionary/base form.
- Discuss whether the normalization could change the meaning of a sentence or document.

16.1 Optional quantitative indicator

For a simple laboratory comparison, I can compute a vocabulary reduction percentage. This is not a complete semantic metric, but it provides a reproducible numerical observation.

```
def reduction_percentage(original, normalized):
    original_unique = len(set(original))
    normalized_unique = len(set(normalized))
    return (1 - normalized_unique / original_unique) * 100


print(
    "Porter vocabulary reduction:",
    round(reduction_percentage(original_words, porter_words), 2), "%"
)


print(
    "Snowball vocabulary reduction:",
    round(reduction_percentage(original_words, snowball_words), 2), "%"
)


print(
    "spaCy lemma vocabulary reduction:",
    round(reduction_percentage(original_words, lemma_words), 2), "%"
)
```

I should report this value together with qualitative examples. Vocabulary reduction alone must not be used to claim that one method is semantically superior.

15. Step 10 – Compare a Short Sentence Before and After Normalization

```
sample = "The students were studying and the researchers studied the results."

sample_doc = nlp(sample)

print("Original:")
print(sample)

print("\nPorter stems:")
print([
    porter.stem(t.text.lower())
    for t in sample_doc if t.is_alpha
])

print("\nSnowball stems:")
print([
    snowball.stem(t.text.lower())
    for t in sample_doc if t.is_alpha
])

print("\nspaCy lemmas:")
print([
    t.lemma_.lower()
    for t in sample_doc if t.is_alpha
])
```

I will use this output to explain the semantic/linguistic quality of each normalization approach.

16. Expected Observation Format

Original	Porter Stem	Snowball Stem	spaCy Lemma	My Observation
studying	studi	studi	study	Lemma is a valid base word; stems are truncated.

studied	studi	studi	study	Stemming and lemmatization reduce variation differently.
processing	process	process	process	All methods may produce an interpretable form here.
systems	system	system	system	Plural normalization is clear.

17. Result Table – To Be Filled by Me

Method	Total Tokens/Words	Unique Forms	Reduction %	Semantic Coherence
Whitespace tokenization	_____	_____	_____	_____
spaCy tokenization	_____	_____	_____	_____
Porter stemming	_____	_____	_____	_____
Snowball stemming	_____	_____	_____	_____
spaCy lemmatization	_____	_____	_____	_____

18. Questions

1. What is the difference between a token and a word?
2. Why does whitespace tokenization give different results from spaCy tokenization?
3. Why can punctuation be useful during tokenization but removed before some downstream tasks?
4. What is the difference between a stem and a lemma?
5. Why can a stemmer produce a string that is not an English word?
6. Why is lemmatization generally more linguistically meaningful than stemming?

7. Why does spaCy require a language model such as en_core_web_sm for linguistic lemmatization?
8. Does lower vocabulary size always mean better NLP preprocessing? Explain.
9. Why does spaCy not use NLTK's Porter stemmer internally?
10. Which method would you choose for a search engine, and which for a linguistically sensitive NLP application? Justify.